



Gymnázium J. K. Tyla

DarkFactory
Agentic and Harness Engineering:
Artificial Intelligence in Practice

THESIS

Author: Patrik Marius, 4.D

Supervisor: Michal Dočekal

Extent: 4682 words / 30839 characters
Extent: 5484 words / 36138 characters

Declaration

I declare that I prepared this specialized thesis independently under the supervision of the supervisor named on the title page. All sources used are listed in the bibliography and information derived from them is cited in the text. I agree that the printed version may be archived at Gymnázium J. K. Tyla and used there as a reference for future student work or for presenting education at GJKT.

In Hradci Králové on

.....

Author signature:

Annotation

[EN] This thesis examines the principles of agentic engineering: effective engineering practices for development with artificial intelligence through agentic systems and the architecture of these systems. The practical contribution of the thesis is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize automation of the development lifecycle, from interpreting requirements in GitHub Issues, through planning, to code development and merging changes. The thesis reflects that current agentic systems cannot be regarded as fully autonomous: language models require deterministic guardrails, context management, and human involvement (*Human-in-the-loop*).

Keywords

Actions , , Agent Harness , Agent Loop , , , , Container , , , DAG , Embedding , Flaky Test , , Guardrail , HITL , Hook , Issue , KV Cache , LLM , Loop Engineering , MCP , Plugins , Progressive Disclosure , , Pull Request , , Skills , Squash , Subagent , , Transformer

Contents

<u>Declaration</u>	2
<u>Annotation</u>	3
<u>Keywords</u>	4
1 <u>Introduction</u>	8
1.1 <u>Motivation and Problem Definition</u>	8
1.1.1 <u>Global Generative AI Adoption</u>	8
1.2 <u>Thesis Objective and Research Questions</u>	9
1.2.1 <u>Main Goal</u>	9
1.2.2 <u>Sub-goals</u>	9
1.2.3 <u>Research Questions</u>	9
1.3 <u>Methodology</u>	9
2 <u>Development Environment and Practices</u>	10
2.1 <u>GitHub</u>	10
2.2 <u>Software Engineering</u>	10
2.2.1 <u>Planning</u>	10
2.3 <u>Version control</u>	10
2.3.1 <u>Git</u>	10
2.3.1.1 <u>Repository Branch</u>	11
2.3.1.2 <u>Branch Merge</u>	12
2.3.1.3 <u>Commit Squashing</u>	12
2.3.1.4 <u>GitHub Issue</u>	12
2.3.1.5 <u>Pull Request</u>	12
2.3.1.6 <u>Required Checks</u>	13
2.3.1.7 <u>Branch Protection</u>	13
2.4 <u>Continuous Integration</u>	13
2.4.1 <u>GitHub Actions</u>	14
2.4.2 <u>Software Container</u>	14
2.4.3 <u>Flaky Test</u>	14
3 <u>Language Models, Chatbots, and Agents</u>	15
3.1 <u>Chatbot</u>	15
3.1.1 <u>ChatGPT</u>	15
3.1.1.1 <u>ChatGPT App Interface</u>	15
3.2 <u>Agent</u>	16
3.3 <u>Large Language Model</u>	17

3.3.1	GPT-5.6	17
3.3.1.1	GPT-5.6 Product Visual	17
3.3.2	Claude Opus 5	17
3.3.2.1	Claude Opus 5 Product Visual	18
3.3.3	DeepSeek-V4.1-Flash	18
3.3.3.1	DeepSeek-V4.1-Flash Product Visual	18
3.3.4	Transformer Architecture	19
3.3.5	Autoregressive Modeling	19
3.3.6	Stochastic Decoding	19
3.3.7	Tokenizer	19
3.3.8	Token	19
3.3.9	Embedding	20
3.3.10	Context Window	20
3.3.11	Key-Value Cache	20
3.3.12	Interaction Turn	20
4	Agentic Engineering	21
4.1	Prompt Engineering	21
4.1.1	System Prompt	21
4.2	Agent Harness	21
4.2.1	Codex	22
4.2.1.1	Codex App Interface	22
4.2.2	Claude Code	22
4.2.2.1	Claude Code Interface	22
4.2.3	Claude Desktop App	23
4.2.3.1	Claude Desktop Interface	23
4.2.4	Sandbox	24
4.2.5	Deterministic Guardrail	24
4.2.6	Human-in-the-loop	25
4.2.7	Plugins	25
4.2.8	ReAct Loop	26
4.2.8.1	Agent Session	27
4.2.8.2	Execution-loop Engineering	27
4.2.8.3	Agent Divergence Pathologies	27
4.2.9	Tool Calling	28
4.2.9.1	Model Context Protocol	28
4.2.10	Skills	29

4.2.10.1	<u>Progressive Disclosure</u>	29
4.2.10.2	<u>Script</u>	29
4.2.10.3	<u>Event Hook</u>	30
4.2.11	<u>Context Engineering</u>	30
4.2.11.1	<u>Context Compaction</u>	30
4.2.11.2	<u>Retrieval-Augmented Generation</u>	30
4.2.11.3	<u>Context Rot</u>	31
4.2.11.4	<u>Semantic Drift</u>	31
4.2.12	<u>Workflow-graph Engineering</u>	32
4.2.12.1	<u>Subagent</u>	32
4.2.12.2	<u>Directed Acyclic Graph</u>	32
5	<u>Results and Discussion</u>	33
6	<u>Conclusion</u>	33
	<u>List of figures and tables</u>	34
	<u>List of appendices</u>	35
	References	36

1 Introduction

1.1 Motivation and Problem Definition

Motivací práce je ukázat, jaké úlohy dokážou současné agentní systémy samostatně provádět a jaká technická vrstva umožňuje převést schopnost jazykového modelu do spolehlivého jednání nad skutečným softwarovým projektem. Generativní AI se během několika let rozšířila do masového používání. Microsoft AI Economy Institute odhaduje, že v prvním čtvrtletí 2026 použilo generativní AI 17,8 % světové populace v produktivním věku. Současně se rychle rozšiřují systémy, které už pouze negenerují odpověď, ale plánují více kroků, pracují se soubory a nástroji, spouštějí příkazy a testy, kontrolují vlastní výstupy a mohou vykonávat dlouhotrvající úlohy. ⁽¹⁾

V softwarovém inženýrství tento posun reprezentují například Codex a Claude Code: jejich oficiální popisy uvádějí práci nad reálnými repozitáři, paralelní agentní úlohy, úpravy souborů, spouštění testů a další činnosti přesahující jednorázové generování kódu. ^(2,3)

Práce proto nezkoumá pouze schopnost modelu vytvořit fragment programu. Zaměřuje se na **Agent Harness** jako systémovou vrstvu, která modelu poskytuje nástroje, stav, kontext, oprávnění, kontrolní smyčku a ověřování výsledků. Právě tato vrstva rozhoduje o tom, zda lze schopnosti modelu využít jako opakovatelný agentní proces.

1.1.1 Global Generative AI Adoption

Grafický podklad dokumentuje rychlost, s níž se generativní AI rozšířila z experimentální technologie do běžného používání. Microsoft AI Economy Institute uvádí pro druhé pololetí roku 2025 celosvětový podíl uživatelů 16,3 %, oproti 15,1 % v prvním pololetí. Novější zpráva pro první čtvrtletí 2026 odhaduje používání generativní AI na 17,8 % světové populace v produktivním věku.

Region	H1 2025 AI Diffusion	H2 2025 AI Diffusion	Change
Global North	22.9%	24.7%	+1.8
Global South	13.1%	14.1%	+1.0
World	15.1%	16.3%	+1.2

Obrázek 1: AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute.

Měřítko adopce neposuzuje schopnosti jednotlivých systémů, ale dokládá jejich rychlé rozšíření do reálného používání. ^(1,4) Výchozí otázkou práce je, co současné agentní systémy skutečně dokážou a jak musí být navržen harness, aby jejich schopnosti bylo možné bezpečně a opakovatelně využít v reálném vývojovém prostředí. ^(1–3)

1.2 Thesis Objective and Research Questions

1.2.1 Main Goal

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2 Sub-goals

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3 Research Questions

- VO1 (Míra automatizace a role člověka): Lze vývojový proces od zadání požadavku (GitHub Issue) po pull request strukturovat tak, aby role vývojáře spočívala výhradně v architektonickém dozoru a schvalování záměru (Human Gate), bez nutnosti ručního psaní rutinního kódu?
- VO2 (Řízení divergence a spolehlivost smyčky): Jakými architektonickými mechanismy lze v harnessu spolehlivě zabránit patologiím modelu (perseveraci, oscilaci a zacyklení v ReAct smyčce)?
- VO3 (Integrita paměti a eliminace sémantického posunu): Jak spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a ztrátě architektonických invariantů při kompresi?

 **Hlubková kritika / Oponentura:** Oponentura k výzkumným otázkám: Otázka VO1 je formulována binárně („Lze vývojový proces strukturovat...“), což svádí k tautologické odpovědi. Rigorózní oponent bude žádat empirické vymezení: Jaké procento rutinních úloh (např. oprava chyby se selhávajícím testem vs. komplexní refaktoring) harness reálně odbaví bez ručního zásahu do kódu? Doporučujeme otázku v obhajobě doplnit o kritérium mezní složitosti úkolu a míry redukce kognitivní zátěže člověka.

1.3 Methodology

Práce používá konceptově-analytický a inženýrský postup: vymezuje nezbytné části agentního systému, popisuje jejich vztahy a ověřuje je na návrhu a realizaci systému DarkFactory. Předmětem práce není trénování neuronových sítí, optimalizace vah ani podrobná matematika modelového učení. **LLM** je zde chápán jako hotová inferenční komponenta. Modelová vrstva je proto popsána pouze v rozsahu nutném pro pochopení agentního běhu, tokenizace, kontextového okna, paměťového stavu inference a nástrojových rozhraní.

Hlavním předmětem zkoumání je konstrukce **Agent Harness** pro autonomní softwarové inženýrství. Jednotlivé mechanismy jsou rozděleny do samostatných konceptů, aby každý pojem měl jedno místo pro definici, popis a shrnutí a aby se stejné vysvětlení neopakovalo v několika kapitolách.

Postup práce má tři kroky: nejprve jsou vymezeny relevantní koncepty a jejich závislosti; následně jsou tyto principy promítnuty do architektury DarkFactory; nakonec jsou vlastnosti výsledného systému posouzeny podle ověřitelných výstupů vývojového procesu. Terminologie zachovává zavedené oborové názvy tam, kde jsou v praxi přesnější než doslovný překlad, přičemž všechny používané termíny jsou vedeny jako kanonické koncepty. Rozsah práce je záměrně soustředěn na agentní systém kolem modelu: model je vstupní inferenční komponenta, zatímco předmětem návrhu a hodnocení je harness a jeho provozní mechanismy.

2 Development Environment and Practices

The set of versioning, planning, integration, and verification practices that form a deterministic environment for agentic software development. ⁽⁵⁾

2.1 GitHub

A platform for hosting Git repositories and coordinating the software-development lifecycle through features such as Issues, Pull Requests, and CI/CD automation. ⁽⁶⁾ GitHub poskytuje nad gitovým repozitářem koordinační vrstvu pro zadání práce, revizi změn a automatizaci. Tyto odpovědnosti jsou v práci dále rozloženy mezi samostatné koncepty Issue, Pull Request a GitHub Actions.

2.2 Software Engineering

The systematic application of engineering principles to the specification, design, implementation, verification, operation, and maintenance of software systems. ⁽⁵⁾

2.2.1 Planning

The process of turning a requirement into an explicit sequence of steps, dependencies, and verification conditions before changes are executed. ⁽⁵⁾

2.3 Version control

The management and tracking of changes to source files and other versioned artifacts so changes can be safely branched, merged, audited, and reverted when necessary. ⁽⁷⁾

2.3.1 Git

A distributed version-control system that records project history, supports branching and merging, and enables deterministic return to earlier repository states. ⁽⁷⁾ Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely

generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém : v kombinaci s platformou .:

- Klíčové komponenty infrastruktur zahrnují:

- Distribuovaný systém Git ⁽⁷⁾: Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - Zadávání a sledování úkolů (*Issues*): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - Revize změn (*Pull Requests*): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - Automatizace (*GitHub Actions*): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

[+] Samotný : zde zůstává vymezen jako distribuovaný systém správy verzí. Hosting a koordinaci repozitáře popisuje , zadání práce *Issue* , revizní integraci *Pull Request* a automatizaci *Actions* .:

2.3.1.1 Repository Branch

A named line of development in version control that allows changes to proceed separately from another history line and later be compared or merged. ⁽⁷⁾ Základním bezpečnostním pravidlem při zapojení autonomních agentů do vývoje je striktní izolace rozpracovaného kódu. Stabilní kód v hlavní větvi (*main*) nesmí být nikdy přímo vystaven experimentům a chybám modelu. Agent proto veškeré úpravy provádí ve vyhrazených pracovních větvích odbočených ze základní linie projektu.

Tento princip přináší následující výhody:

- - Ochrana produkční větve [+] Ochrana produkční větve ⁽⁷⁾: Hlavní větev (*main*) reprezentuje stabilní, otestovaný stav připravený k nasazení. Přímé zapisování do této větve je zakázáno jak lidským vývojářům, tak autonomním agentům.
- Dedikovaná větev pro každý úkol: Agent pro každé zadání dynamicky vytvoří novou samostatnou větev (např. *task/123-oprava-parseru* či *agent/feature-auth*).
- Izolace chyb a mezystavů: Případné syntaktické chyby, dočasné nefunkční stavy ani neúspěšné hypotézy neovlivňují stabilitu hlavní větve ani práci ostatních vývojářů v týmu.
- Bezpečné zahození nezdařených běhů: Pokud se agent dostane do slepé uličky nebo vyčerpá přidělený rozpočet kroků, celou větev lze smazat jedním příkazem bez jakýchkoliv následků pro zbytek repozitáře.

Pokud se hlavní větev během práce agenta posune dopředu v důsledku jiné aktivity v repozitáři, pracovní větev agenta se musí před dokončením zaktualizovat (`git rebase` nebo `git merge`), aby byla zajištěna bezkonfliktní integrace.

2.3.1.2 Branch Merge

A version-control operation that combines changes or history from two lines of development into a common resulting state; conflicts require explicit resolution. ⁽⁷⁾ Způsob, jakým se změny z pracovní větve začlení do větve hlavní, má zásadní dopad na dlouhodobou udržitelnost a čitelnost repozitáře. Autonomní agent při řešení úlohy obvykle postupuje iterativní metodou pokus-omyl: upraví soubor, spustí testy, odhalí překlep a provede další drobný commit. V pracovní větvi tak vzniká dlouhá sekvence pomocných a experimentálních záznamů.

Zatímco klasický merge commit přenese do hlavní větve veškeré dílčí commity a rebase je lineárně přeskládá, v agentním vývoji se jako optimální strategie uplatňuje - Commit and

Merge: `[+ ] Commit and Merge` ⁽⁷⁾:

- Sloučení mezikroků (Squash): Všechny commity z pracovní větve jsou spojeny do jediného nového commitu, který je vložen do `main`.
- Eliminace interního šumu: Pomocné commity vzniklé při ladění testů se do hlavní větve vůbec nedostanou; historie projektu zůstává čistá a přehledná podle pravidla: jeden úkol = jeden commit.
- Atomický návrat změn (`git revert`): Pokud se v budoucnu ukáže, že začleněná úprava zanesla do produkce nečekanou vadu, lze celý úkol vrátit jediným atomickým příkazem bez nutnosti rozplétat desítky dílčích mezikroků.

2.3.1.3 Commit Squashing

An operation that replaces multiple consecutive commits with one aggregate commit, commonly to simplify history before integrating changes. ⁽⁷⁾

2.3.1.4 GitHub Issue

A structured repository record for a request, task, or defect that can carry a description, discussion, labels, assignments, and links to code changes. ⁽⁶⁾ V agentním vývojovém procesu může Issue fungovat jako explicitní vstupní specifikace úlohy. Odděluje zadání od samotné implementace a poskytuje stabilní referenční bod pro plán, pull request i následnou revizi.

2.3.1.5 Pull Request

A formal proposal to integrate changes from one repository branch into another, providing a place for automated checks, human review, and discussion of the proposed changes. ⁽⁷⁾ - Pull request (PR) představuje stěžejní komunikační uzel mezi autonomním agentem a lidským inženýrem. Jedná se o formální žádost o začlenění navržených změn z pracovní větve do větve hlavní. V tomto bodě se plně uplatňuje princip zapojení člověka do smyčky (Human-in-the-loop): `[+ ] Pull Request` : — [cz] Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako místo pro automatizované kontroly, lidskou revizi a

diskusi nad navrženými úpravami. ⁽⁷⁾ V tomto bodě se plně uplatňuje princip **HITL** : agent kód samostatně navrhne a otestuje, avšak konečné rozhodnutí o jeho přijetí náleží vývojáři. Rozhraní pull requestu integruje všechny podstatné informace na jednom místě:

- Rádkový diff: Vizuální srovnání původního a nového stavu, kde jsou jasné barevně odlišeny přidané, změněné a smazané řádky.
- Strukturovaný souhrn změn: Agent v popisu PR srozumitelně shrne, jaké úpravy provedl, jakou logiku zvolil a na které původní issue reagoval.
- Výsledky automatických kontrol: Přehled stavu automatizovaných testů a linterů z GitHub Actions (zelený či červený indikátor).
- Revizní diskuse: Možnost vývojáře přidávat komentáře k libovolnému řádku kódu, klást doplňující dotazy nebo vyžadovat přepracování konkrétních částí.

Lidský vývojář v roli revizora (Reviewer) posuzuje celkový architektonický záměr a rozhoduje o schválení, vrácení k dopracování, či zamítnutí pull requestu.

2.3.1.6 Required Checks

Automated checks whose successful completion is a mandatory condition for accepting or merging a change. ⁽⁸⁾ K tomu, aby byla kontinuální integrace efektivní, nestačí testy pouze spouštět — jejich úspěšné dokončení musí být systémově vynuceno.

- - Požadované kontroly (Required Checks): [+]Požadované kontroly (Required Checks ⁽⁸⁾):
Seznam úloh v GitHub Actions, které musí skončit explicitním úspěchem (zelený stav), aby bylo technicky možné pull request sloučit:
 - Statická analýza a linter: Kontrola dodržení kódového stylu, odhalování mrtvého kódu a základních syntaktických prohřešků.
 - Typová kontrola a build: Jistota, že kód lze bez chyb zkompileovat a že typový systém nezaznamenal nekonzistence.
 - Automatizované testy: Úspěšný průchod jednotkových i integračních testů ověřujících požadované chování.
- Pravidlo deterministického výsledku: Každá kontrola musí skončit jednoznačným výsledkem; tiché přeskočení testu nebo nejednoznačný stav sloučení zablokuje.

2.3.1.7 Branch Protection

Repository rules that restrict direct changes to protected branches and enforce approvals, checks, or other conditions before merging. ⁽⁷⁾ GitHub poskytuje pravidla ochrany větví - **(Branch Protection Rules)** [+]**(Branch Protection Rules ⁽⁷⁾)**, která zabraňují začlenění neověřeného kódu do stabilní větve `main`.

- Povinné schválení člověkem: Požadavek na explicitní autorizaci kódu lidským vývojářem dříve, než GitHub povolí sloučení do produkční větve.

2.4 Continuous Integration

A development practice in which changes are integrated frequently and automatically verified by builds, tests, and other checks so integration failures are detected early. ⁽⁸⁾ Samotný jazykový model kód pouze generuje na základě statistických závislostí v trénovacích datech; nemá schopnost vnitřně ověřit, zda je vytvořený program syntakticky bezchybný a funkčně

správný. Nezastupitelnou roli objektivního arbitra správnosti proto plní kontinuální integrace (CI) ⁽⁸⁾.

- V rámci platformy GitHub zajišťuje kontinuální integraci automatizační platforma

Actions :

- Izolované běhové prostředí (Container): Workflow běží v deklarovaném runner prostředí se stanovenými nástroji a závislostmi, čímž se omezuje závislost na lokálním stavu počítače vývojáře.
- Automatická exekuce: Integrační pipeline se automaticky spouští při každém pushi do pracovní větve i při otevření pull requestu.
- Deterministická zpětná vazba pro agenta: Pokud překlad nebo testy selžou, chybový protokol z terminálu je předán zpět do kontextu agenta, který na jeho základě provede informovanou opravu kódu.

[+] Konkrétní automatizační platformu popisuje **Actions** , izolaci běhu **Container** a problematiku nedeterministických selhání **Flaky Test** . Koncept CI zde zůstává zaměřen na integrační kontrakt a strojově ověřitelnou zpětnou vazbu.

-

 **Hlubková kritika / Oponentura:** Nestálost testů (Flaky Tests) v integračních bězích: Společně se na automatické testy v CI naráží na problém nestálých testů (*flaky tests*), které občas selžou kvůli časování, síťové odezvě či asynchronním stavům, aniž by kód obsahoval chybu. Pokud agent narazí na takto náhodně selhávající test, může začít nesmyslně upravovat správný kód ve snaze chybu odstranit. CI pipeline proto musí nestálé testy minimalizovat nebo umožnit automatické opakování selhaného běhu v čistém prostředí.

[+] Riziko nestálých testů je vyčleněno do samostatného konceptu **Flaky Test** .

2.4.1 GitHub Actions

GitHub's automation platform for running declared workflows and their jobs in response to repository events or manual dispatch. ⁽⁹⁾ GitHub Actions spouští deklarovanou workflow v reakci na události repozitáře. V této práci představuje konkrétní automatizační prostředí, které realizuje CI kontroly a další repozitářové procesy.

2.4.2 Software Container

An isolated user-space runtime packaging an application and its dependencies while sharing the host operating-system kernel; its security isolation depends on the implementation and configuration. ⁽¹⁰⁾ Kontejner může CI běhu poskytnout opakovatelné uživatelské prostředí s deklarovanými nástroji a závislostmi. Tím omezuje vliv lokální konfigurace vývojářského počítače, aniž by sám o sobě zaručoval úplnou bezpečnostní izolaci.

2.4.3 Flaky Test

A test that can alternately pass and fail on the same code because of nondeterminism, timing, environment dependence, or external services. ⁽⁸⁾ Nestálé testy oslabují roli CI jako

deterministické zpětné vazby. Agent může náhodné selhání mylně interpretovat jako regresi a začít měnit správný kód; proto je vhodné zdroje nedeterminismu omezovat a podezřelé běhy ověřovat opakováním v čistém prostředí.

3 Language Models, Chatbots, and Agents

Tato skupina konceptů rozlišuje samotný jazykový model od produktových rozhraní a agentních systémů, které model používají. **LLM**, a označují různé systémové vrstvy. Toto rozlišení zabraňuje zaměňování schopností modelu s funkcemi, které ve skutečnosti poskytuje aplikační nebo agentní harness. Další sekce proto popisují model, jeho bezprostřední inferenční mechanismy a nadřazené systémy odděleně.

3.1 Chatbot

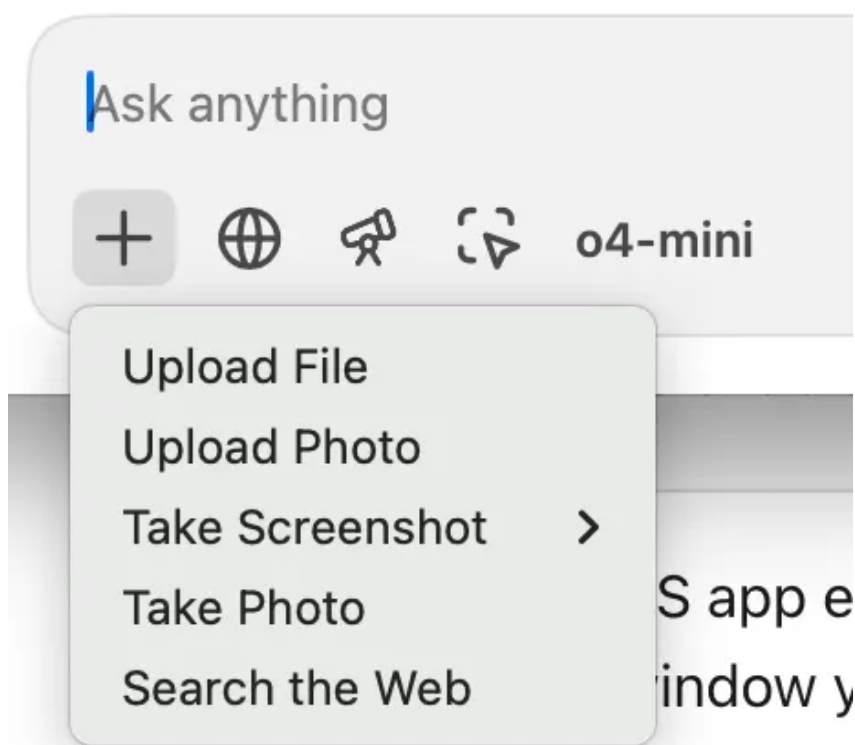
Chatbot je aplikační systém, který zpřístupňuje jazykový model prostřednictvím konverzační interakce. Na rozdíl od samotného modelu může chatbot přidávat historii konverzace, multimodální vstupy, práci se soubory, webové vyhledávání nebo další nástroje. Tyto funkce patří aplikační vrstvě, nikoli modelu samotnému.

3.1.1 ChatGPT

ChatGPT je konverzační produkt OpenAI zpřístupňující modely prostřednictvím chatového a aplikačního rozhraní. Jako příklad chatbota odděluje modelovou vrstvu od uživatelského produktu: rozhraní přidává práci se soubory, obrazem, webem a dalšími nástroji, které samotný model neposkytuje.

3.1.1.1 ChatGPT App Interface

Oficiální snímek rozhraní aplikace ChatGPT pro macOS. Snímek ukazuje vstupní rozhraní a nabídku práce se soubory, obrazem a webem.



Obrázek 2: Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.

Příklad uživatelského povrchu nad jazykovým modelem.⁽¹¹⁾ ChatGPT ilustruje rozdíl mezi jazykovým modelem a aplikačním systémem, který model obaluje.⁽¹¹⁾ Chatbot je produktová vrstva nad modelem; agentní systém se od něj odlišuje především řízenou prováděcí smyčkou a schopností samostatně vykonávat akce.

3.2 Agent

A software system driven by a language model and equipped with tools that independently plans, observes its environment, and performs multi-step actions toward a specified engineering goal. ⁽¹²⁾ : — [cz] Systém založený na jazykovém modelu určený primárně k interakci s uživatelem prostřednictvím konverzačního rozhraní.. - : — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného inženýrského cíle.. [+] : — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného inženýrského cíle. ^(12, 13). Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- Konverzační chatbot:
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- Autonomní agent:
 - Je vybaven sadou výkonných nástrojů (tools) pro práci s repozitářem.

- ▶ Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
- ▶ Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

3.3 Large Language Model

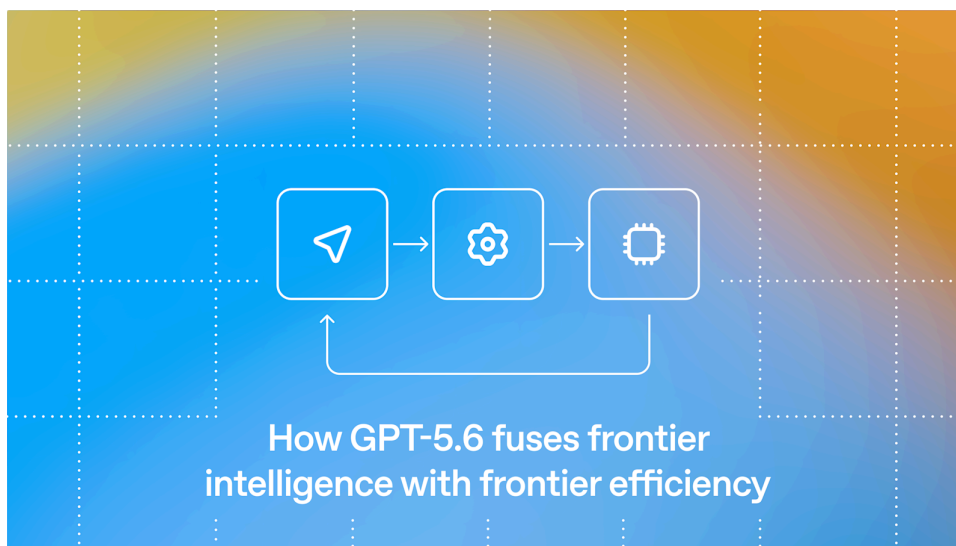
Velký jazykový model je neuronový model pro zpracování a generování posloupností tokenů; současné modely tohoto typu typicky používají **Transformer**. Při generování model opakovaně odhaduje další token na základě dosavadního kontextu. Samostatné koncepty dále popisují tokenizaci, reprezentaci dat a omezení kontextu. Funkce jako práce se soubory, spouštění příkazů nebo správa úloh nejsou vlastnostmi jazykového modelu; poskytuje je nadřazený aplikační nebo agentní systém.

3.3.1 GPT-5.6

GPT-5.6 je modelová řada OpenAI vydaná v roce 2026 ve variantách Sol, Terra a Luna. Jde o příklad současného jazykového modelu, který je nasazován jako inferenční komponenta v produktech včetně ChatGPT a Codexu; aplikační schopnosti těchto produktů však vznikají až propojením modelu s jejich harnessy.

3.3.1.1 GPT-5.6 Product Visual

Oficiální produktový vizuál modelové řady GPT-5.6. Obrazový podklad je převzat z oficiálního vydání modelu OpenAI.



Obrázek 3: GPT-5.6. Zdroj: OpenAI.

Vizuální identifikace příkladu jazykového modelu.⁽¹⁴⁾ Příklad odděluje modelovou vrstvu od agentního systému, který model používá.⁽¹⁴⁾

3.3.2 Claude Opus 5

Claude Opus 5 je model společnosti Anthropic vydaný v červenci 2026. Model je příkladem současné modelové vrstvy používané v širších agentních produktech. Samotný model není

totožný s Claude Code ani desktopovou aplikací Claude; ty přidávají nástroje, stav a orchestrace.

3.3.2.1 Claude Opus 5 Product Visual

Oficiální produktový vizuál modelu Claude Opus 5. Obrazový podklad je převzat z oficiálního oznámení společnosti Anthropic.



Obrázek 4: Claude Opus 5. Zdroj: Anthropic.

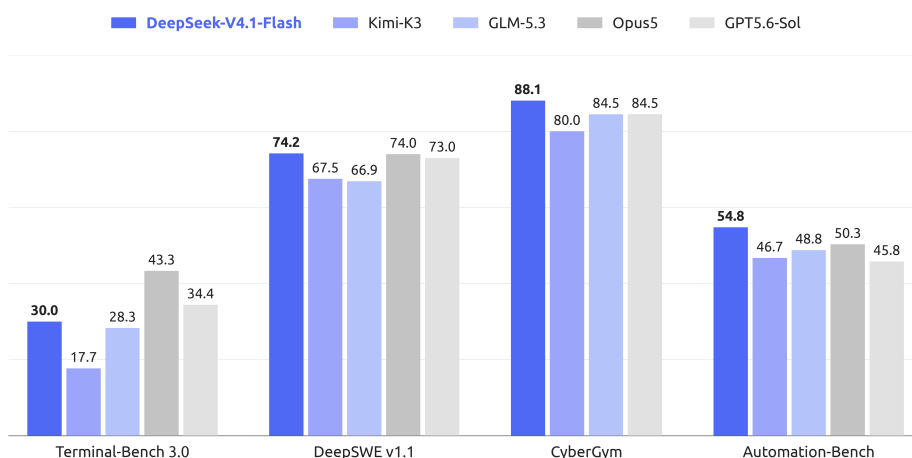
Vizuální identifikace příkladu jazykového modelu.⁽¹⁵⁾ Příklad zdůrazňuje rozdíl mezi modelem a agentním harness-em.⁽¹⁵⁾

3.3.3 DeepSeek-V4.1-Flash

DeepSeek-V4.1-Flash je multimodální model DeepSeek vydaný v září 2026. Jde o příklad odlišné současné modelové architektury: DeepSeek jej popisuje jako 552B MoE s asymetrickým Causal Encoder–Decoder uspořádáním a nativním vizuálním vstupem.

3.3.3.1 DeepSeek-V4.1-Flash Product Visual

Oficiální produktový vizuál modelu DeepSeek-V4.1-Flash. Obrazový podklad je převzat z oficiálního vydání společnosti DeepSeek.



Obrázek 5: DeepSeek-V4.1-Flash. Zdroj: DeepSeek.

Vizuální identifikace příkladu jazykového modelu.⁽¹⁶⁾ Příklad ukazuje, že pojem jazykového modelu v agentním systému zahrnuje různé konkrétní modelové rodiny a architektury.⁽¹⁶⁾ Jazykový model je inferenční komponenta. V této práci je důležité především jeho rozhraní a omezení, protože provozní chování agenta vzniká až propojením modelu s harness-em.

3.3.4 Transformer Architecture

A neural-network architecture based on attention mechanisms that models relationships among sequence elements and underlies most contemporary large language models.⁽¹⁷⁾ Současné velké jazykové modely jsou typicky realizovány transformerovou architekturou. Pro generování textu se často používá dekodérové uspořádání, které nad dosavadní sekvenci vytváří reprezentace potřebné k odhadu následujícího tokenu.

3.3.5 Autoregressive Modeling

A sequential modeling approach in which a model predicts the next token conditioned on preceding tokens in the current sequence.⁽¹⁷⁾ Autoregresivní generování probíhá iterativně: model zpracuje dosavadní posloupnost tokenů, vypočítá distribuci pravděpodobnosti následujícího tokenu a po jeho výběru celý krok opakuje nad rozšířenou posloupností.

3.3.6 Stochastic Decoding

Selection of output tokens from a model probability distribution in a way that can yield different valid continuations for the same input.⁽¹⁷⁾ Pravděpodobnostní dekódování znamená, že shodný vstup nemusí vždy vytvořit totožný výstup. V agentním systému proto nelze provozní spolehlivost opřít pouze o model; kritická pravidla a stavové přechody musí vynucovat deterministická vrstva harnessu.

3.3.7 Tokenizer

A component that maps text or another input into a sequence of tokens and token identifiers and, where supported, performs the reverse decoding transformation.⁽¹⁸⁾

3.3.8 Token

A discrete unit processed by a language model. A token corresponds to an entry in the tokenizer vocabulary and is represented by a numeric identifier; it need not correspond to a whole word.⁽¹⁸⁾ Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- Tokeny a tokenizér (): Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér - (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE) [+](nejčastěji na bázi algoritmu Byte Pair Encoding, BPE⁽¹⁸⁾) .

- **Embedding**: — [cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi. ⁽¹⁹⁾ (např. vektorová analogie král — muž + žena $\approx$ královna).
- Jazyková asymetrie tokenizace: Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

3.3.9 Embedding

A multidimensional vector representation of tokens or other data in which numerical relationships between vectors capture useful semantic relationships between representations. ⁽¹⁹⁾

3.3.10 Context Window

The maximum token-sequence span a model can include in active context during one inference run. In practice it limits the combined instructions, history, tool outputs, and other data supplied to the model. ⁽²⁰⁾

3.3.11 Key-Value Cache

A cache of previously computed key and value vectors in transformer attention layers that reduces repeated computation of earlier tokens during autoregressive generation. ⁽²¹⁾

3.3.12 Interaction Turn

One discrete unit of interaction in a conversational or agentic protocol, such as a user message, model response, or separately recorded tool result. ⁽²²⁾ Interakce mezi modelem, uživatelem a okolním vývojovým prostředím neprobíhá spojitě, nýbrž v diskrétních krocích označovaných jako tahy (*turns*). Každý tah představuje jednu ucelenou výměnu zprávy, na niž systém reaguje.

Životní cyklus tahů a správa paměti zahrnují:

- Typy tahů v agentní smyčce:
 - Tah uživatele či prostředí (*User Turn*): Nové zadání úkolu nebo vnější událost.
 - Tah modelu (*Model Turn*): Vygenerovaná odpověď nebo strukturovaný požadavek na spuštění nástroje.
 - Tah nástroje (*Tool Execution Turn*): Zpětné hlášení výsledku exekuce (výpis souboru, výstup kompilátoru).
- - Správa KV cache (KV Cache) [+] Správa KV cache (KV Cache ^(21,23)) : Aby inferenční engine nemusel při každém novém tahu přepočítávat celou historii od začátku, ukládá mezivýpočty klíčů a hodnot matic pozornosti do paměti.
- - Kontextové okno () [+] Kontextové okno (⁽¹⁷⁾) : Pevně limitovaná kapacita paměti modelu. Tento strop je dán hardwarovými limity GPU akceleratorů a kvadratickou složitostí plné pozornosti ($O(N^2)$ vzhledem k délce sekvence).

4 Agentic Engineering

An engineering discipline focused on designing, orchestrating, and operating agentic systems around language models, including tools, context, execution loops, guardrails, and human oversight. ⁽¹³⁾

4.1 Prompt Engineering

An engineering discipline for systematically designing, structuring, and optimizing instructions and system prompts to guide and constrain language-model behavior. ⁽²⁴⁾ - **Základní chování agenta vymezuje systémový prompt ⁽²⁴⁾, který definuje jeho identitu, sadu dostupných nástrojů a provozní mantinely. [+]** : slouží k systematickému návrhu instrukcí, které řídí chování agenta. Základní instrukce jsou obvykle součástí : — [cz] Instrukční vrstva s vysokou prioritou, která vymezuje roli, chování, dostupné prostředky a provozní mantinely jazykového modelu nebo agenta. ⁽²⁴⁾, který vymezuje roli agenta, dostupné nástroje a provozní mantinely. Při formulaci těchto pravidel však vývojáři narážejí na specifickou vlastnost autoregresivních modelů — problematické zpracování zákazů a negativních instrukcí.

Příčiny a inženýrská řešení tohoto jevu:

- Úskalí negativních instrukcí: Žákazy formulované negací (např. „nemazat existující testy“) modely často porušují, protože matice pozornosti (QK^T) asociativně aktivuje zakázaný pojem dříve, než autoregresní proces uplatní logický operátor negace.
- Afirmativní formulace: Pravidla je nutné formulovat pozitivně — namísto výčtu zákazů vymezit přesný postup a povolené mantinely chování.
- - **Deterministická ochrana v harnessu: Kde nestačí prompt, musí zasáhnout kód agent harnessu — například zpřístupněním testovacích souborů pouze pro čtení nebo zablokováním destruktivních operací na úrovni systémového volání. [+] Guardrail** : odděluje programově vynucené bezpečnostní a procesní podmínky od instrukcí, jejichž dodržení závisí na chování modelu.

4.1.1 System Prompt

A high-priority instruction layer that defines the role, behavior, available capabilities, and operating constraints of a language model or agent. ⁽²⁴⁾

4.2 Agent Harness

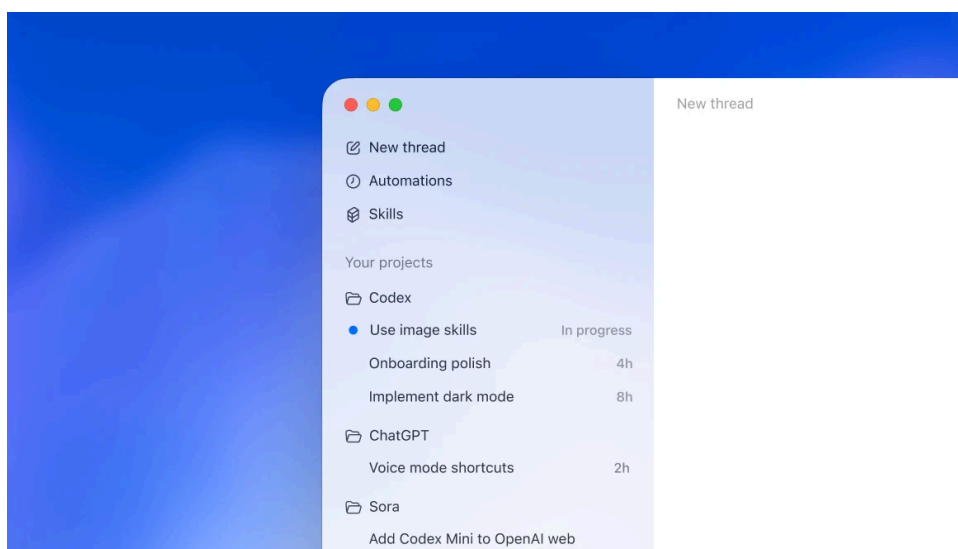
Agentní harness je aplikační a orchestrační vrstva, která propojuje model s prostředím a řídí jeho opakované jednání nad stavem a nástroji. Harness sestavuje kontext, zpřístupňuje a omezuje nástroje, spravuje stav sezení, vyhodnocuje výsledky akcí a rozhoduje o pokračování nebo ukončení prováděcí smyčky. Tím odděluje pravděpodobnostní rozhodování modelu od deterministických pravidel systému. Ústředním prováděcím mechanismem je **Agent Loop** .

4.2.1 Codex

Codex je agentní vývojový systém OpenAI určený pro samostatné plnění softwarově-inženýrských úloh. Desktopová aplikace umožňuje řídit více agentů paralelně, oddělovat jejich práci do vláken a worktree, kontrolovat diffy a delegovat dlouhotrvající úlohy. Tím představuje praktický příklad harnessu, který rozšiřuje model o stav, nástroje, izolaci a pracovní postup.

4.2.1.1 Codex App Interface

Oficiální produktový snímek aplikace Codex. Snímek dokumentuje specializované rozhraní pro práci s agentními úlohami a dovednostmi.



Obrázek 6: Aplikace Codex. Zdroj: OpenAI.

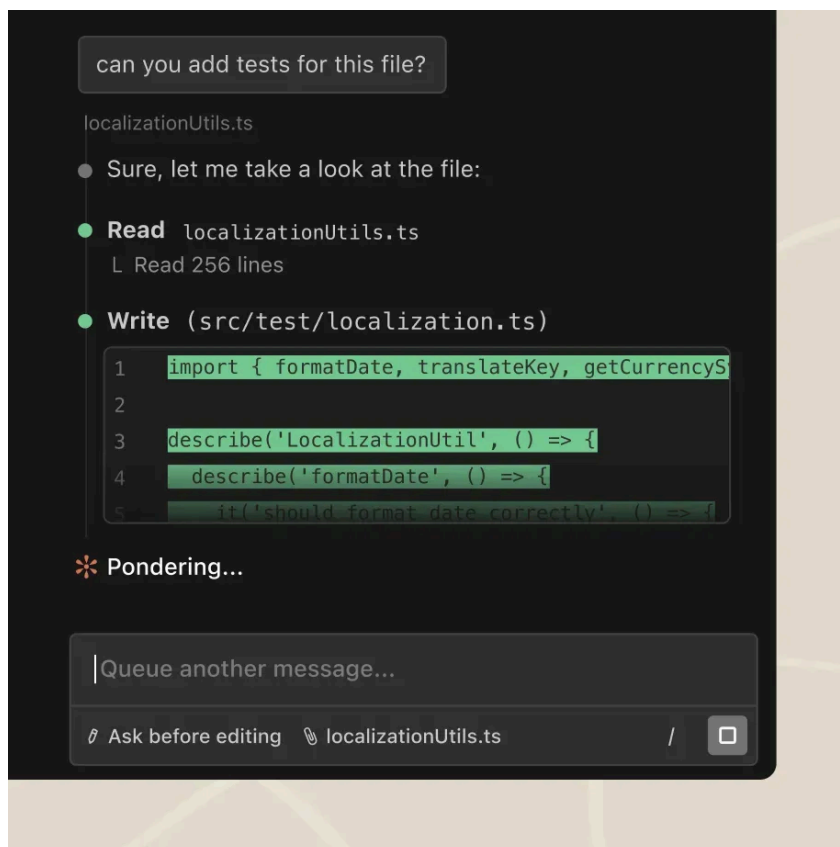
Vizuální příklad agentního vývojového prostředí.⁽²⁾ Codex ukazuje posun od generování fragmentů k řízenému agentnímu vývojovému procesu.⁽²⁾

4.2.2 Claude Code

Claude Code je agentní vývojové prostředí společnosti Anthropic dostupné v terminálu, IDE, na webu a v desktopové aplikaci. Systém může plánovat změny, číst a zapisovat soubory, spouštět příkazy a testy, používat Git a MCP a vytvářet pull requesty. Tyto schopnosti vznikají kombinací modelu s nástrojovou a stavovou vrstvou harnessu.

4.2.2.1 Claude Code Interface

Oficiální snímek rozhraní Claude Code při práci se zdrojovým kódem. Snímek zobrazuje čtení souboru, zápis změny a průběžný stav agentní úlohy.



Obrázek 7: Claude Code při práci se souborem. Zdroj: Anthropic.

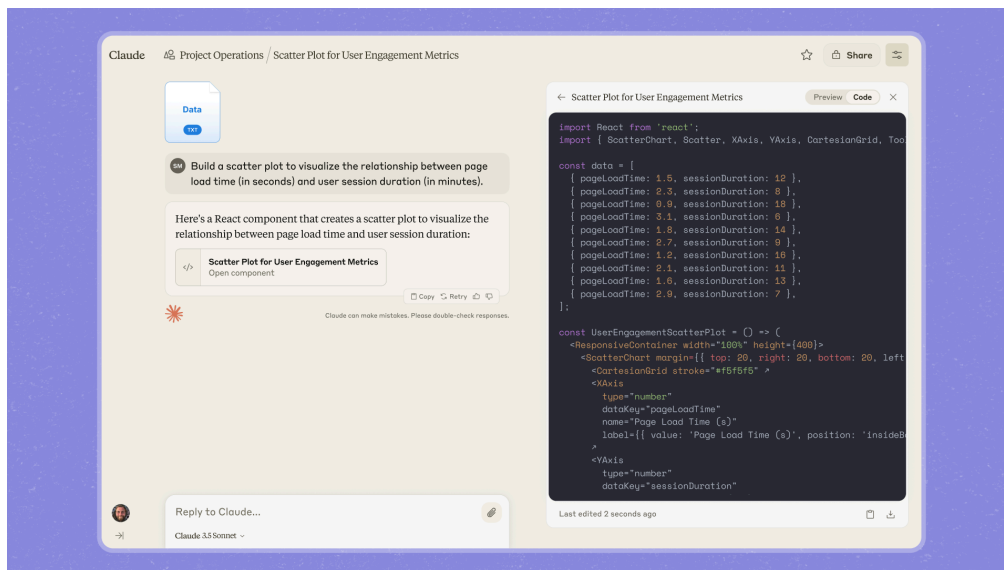
Vizuální příklad nástrojově řízeného kódovacího agenta.⁽³⁾ Claude Code je konkrétní příklad agentního harnessu propojeného s reálným vývojovým prostředím.⁽³⁾

4.2.3 Claude Desktop App

Claude Desktop je desktopová aplikační vrstva nad modely Claude, která propojuje konverzaci s lokálními soubory, aplikacemi, webem a dalšími schopnostmi. Anthropic popisuje desktopovou aplikaci jako prostředí, v němž může Claude se souhlasem uživatele přímo číst, upravovat a ukládat soubory a pracovat s dalšími aplikacemi. Jde proto o širší příklad harnessu než samotný chatbot.

4.2.3.1 Claude Desktop Interface

Oficiální snímek aplikačního rozhraní Claude s pracovním artefaktem. Snímek ukazuje, že aplikační vrstva může vedle konverzace zpřístupnit soubory, nástroje a samostatný pracovní povrch.



Obrázek 8: Aplikační rozhraní Claude. Zdroj: Anthropic.

Vizuální příklad desktopového aplikačního harnessu.⁽²⁵⁾ Desktopová aplikace ukazuje, jak harness rozšiřuje model o oprávnění, kontext a akce v uživatelském prostředí.⁽²⁵⁾ Schopnosti agentního systému proto nelze připsat pouze modelu: vznikají souhrou modelu a harnessu, který mu poskytuje prostředí pro řízené jednání.

4.2.4 Sandbox

A constrained runtime environment intended to isolate executing code and its permissions from the host system.⁽²⁶⁾ Přímé spouštění kódu (*Code Execution*) umožňuje agentovi generovat skripty (bash, Python), které harness spouští v izolovaném terminálu. Tento model poskytuje maximální flexibilitu pro softwarový vývoj, avšak vyžaduje nekompromisní bezpečnostní izolaci.

🔥 Hlubková kritika / Oponentura: Iluzorní bezpečnost pískoviště: Přímé spouštění netestovaného syntetického kódu v běžném Docker kontejneru nelze považovat za plnohodnotnou bezpečnostní hranici (*security boundary*). Přístup k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky environmentálních tajností (GitHub tokeny, API klíče k LLM) přes skryté síťové kanály a kompromitaci CI infrastruktury. Pro bezpečný produkční provoz je nezbytná formální izolace na bázi microVM - (např. AWS Firecracker, gVisor) [+](např. AWS Firecracker⁽²⁶⁾, gVisor) a striktní izolace síťových jmenných prostorů.

4.2.5 Deterministic Guardrail

Guardrail je programově vynucené omezení nebo kontrola, která neponechává kritické provozní pravidlo pouze na pravděpodobnostním rozhodnutí modelu. Pokud samotná instrukce v promptu neposkytuje dostatečnou záruku, harness může pravidlo vynutit deterministicky, například omezením přístupových práv, validací parametrů nástroje nebo odmítnutím nepovoleného stavového přechodu. Kritická bezpečnostní a procesní pravidla

patří do deterministicky vynucované vrstvy harnessu, nikoli pouze do textových instrukcí modelu.

4.2.6 Human-in-the-loop

A design pattern in which a human operator remains part of the system's decision process through approval gates (Human Gates), especially before consequential or irreversible system operations. ⁽²⁷⁾ Základním principem navrženého řešení není nekritická plná autonomie, nýbrž - **efektivní kooperace člověka a stroje** [+] **efektivní kooperace člověka a stroje** (**HITL** ⁽²⁷⁾). Autonomnímu systému náleží mechanické a rutinní úkony, zatímco klíčová architektonická a nevratná rozhodnutí zůstávají plně pod kontrolou vývojáře.

Řízení lidského dohledu staví na těchto pilířích:

- Lidské schvalovací brány (*Human Gates*): Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora:
 - 1. brána (Záměr a plán): Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód v souborech.
 - 2. brána (Sémantická revize): Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
- Prevence únavy z revizí (*Review Fatigue*): Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- Dohledatelnost původního zadání: Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- Transparentnost selhání a deterministická eskalace: Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

 **Hlubková kritika / Oponentura:** Kognitivní limity lidské schvalovací brány (*Review Fatigue*): Spoléhání se na finální sémantickou kontrolu diffu v pull requestu naráží na lidské kognitivní limity. Výzkumy prokazují, že u rozsáhlých diffů (nad 300–400 řádků) dramaticky klesá hloubka lidské pozornosti — vývojář kód pouze zběžně prohlédne a spoléhá na zelenou fajfku z CI. Aby byla lidská brána efektivní, harness musí diffy rozkládat do sémanticky sevřených mikrokroků, generovat interaktivní vysvětlení netriviálních rozhodnutí a explicitně zvýrazňovat změny v kritických architektonických komponentách.

4.2.7 Plugins

Programmatic extension modules running directly in the harness environment that extend its execution core with specialized system adapters, tool drivers, and deterministic hooks. ⁽²⁸⁾

Plugin rozšiřuje samotný harness programovým modulem. Na rozdíl od instrukční doved-

nosti tak může přidávat systémové adaptéry, ovladače nástrojů nebo deterministické zásahy přímo do běhové vrstvy.

4.2.8 ReAct Loop

Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (*Reasoning + Acting*), v němž model střídá rozhodování, volání nástrojů a vyhodnocování výsledků z běhového prostředí. Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

- V každé iteraci agentní smyčky harness zajišťuje tyto klíčové funkce:

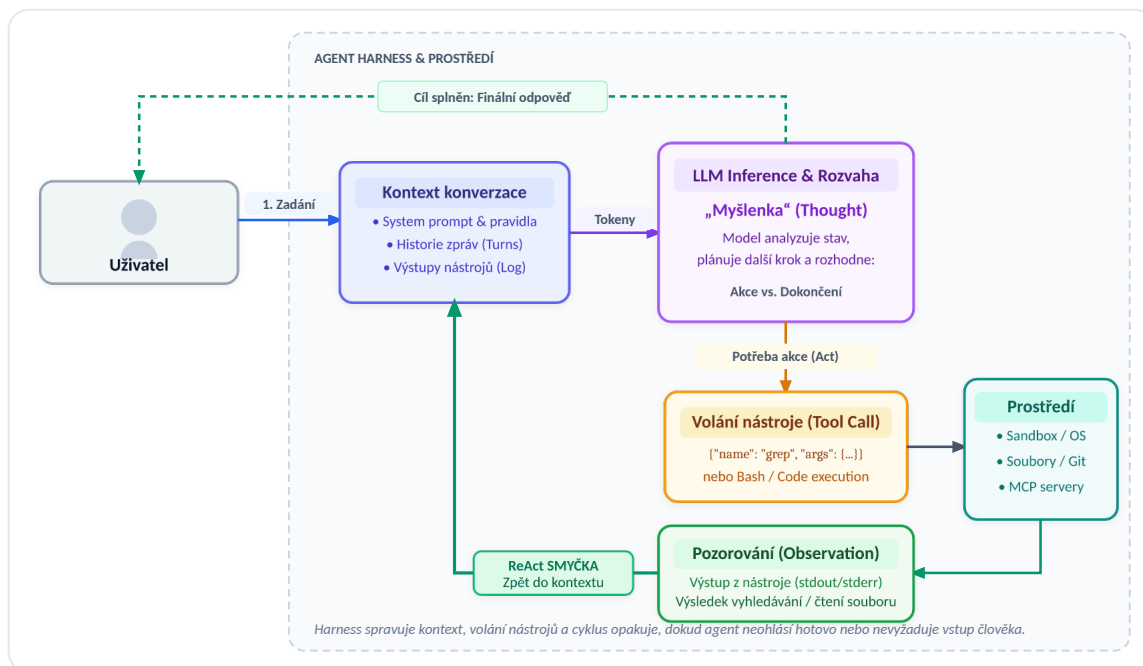
- Inicializace a správa sezení: Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- Běhové prostředí nástrojů: Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.
- Řízení stavových přechodů a vynucování mantinelů (*Loop Engineering*): Dohled nad dodržováním procesních pravidel, detekce a zastavení uvíznutých běhů a vynucování lidských schvalovacích bran.

[+] Provozní odpovědnosti smyčky jsou odděleny do samostatných konceptů: *Tool Calling*, *Loop Engineering* a *Guardrail*. Samotná agentní smyčka zde popisuje jejich iterativní koordinaci.

Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*) ⁽²²⁾, který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 9:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.



Obrázek 9: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Agentní smyčka koordinuje opakované rozhodování modelu s vykonáváním akcí; její provozní spolehlivost proto závisí na řízení stavu, nástrojů, rozpočtů a podmínek ukončení.

4.2.8.1 Agent Session

Agentní sezení je ohraničený běh agenta se sdíleným stavem, systémovými instrukcemi, pracovním kontextem, historií tahů a provozními rozpočty. Správa agentního sezení zahrnuje sestavení systémového promptu, načtení relevantního kontextu repozitáře, průběžné uchování stavu a sledování rozpočtů, například spotřeby tokenů nebo počtu iterací. Sezení poskytuje jednotku životního cyklu, ve které lze konzistentně spravovat stav, kontext a zdrojové limity jednoho agentního běhu.

4.2.8.2 Execution-loop Engineering

Inženýrství prováděcí smyčky je návrh a řízení stavových přechodů, podmínek ukončení, rozpočtů, opakování, eskalací a vazby mezi rozhodováním modelu a nástroji. Inženýrství prováděcí smyčky odděluje provozní řízení od samotného modelového rozhodování. Patří sem stavové přechody, podmínky ukončení, rozpočty běhu, opakování po selhání, detekce uvíznutí a pravidla pro eskalaci nebo lidské schválení. Loop engineering převádí otevřenou agentní iteraci na provozně ohraničený proces s explicitními pravidly pokračování, selhání a ukončení.

4.2.8.3 Agent Divergence Pathologies

Divergence je třída selhání agentní smyčky, při níž se iterativní běh vzdaluje cíli například perseverací, oscilací nebo nekontrolovanou spotřebou zdrojů. - **Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním.** [+]**Ponechání**

jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním ⁽²⁹⁾. V důsledku autoregresivní povahy se v kontextu snadno vytvoří pravděpodobnostní atraktor, který model uvězní v neproduktivním cyklu.

Mezi typické patologie patří:

- Perseverace a zacyklení: Opakované volání identického nástroje se stejnými neplatnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy.
- Oscilace a těkání (*Thrashing*): Střídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije modul A).
- Nekontrolovaná spotřeba zdrojů (*Context Runaway*): Rychlé vyčerpání kontextového okna i finančního rozpočtu na volání API bez dosažení cíle.

Divergenci nelze řešit pouze lepším promptem; harness musí rozpoznávat neproduktivní trajektorie a omezovat jejich pokračování.

4.2.9 Tool Calling

A mechanism through which a model structurally asks a harness to execute an external action or function with validated parameters. ⁽³⁰⁾ Aby mohl agent provádět reálné inženýrské operace, musí mu agent harness zpřístupnit systémové nástroje. Způsob, jakým jsou nástroje modelům předkládány, zásadně ovlivňuje ergonomii vývoje i bezpečnost celého systému.

- Strukturované volání nástrojů (*Tool / Function Calling*) [+]Strukturované volání nástrojů (*Tool / Function Calling*) ⁽³⁰⁾ používá vstupy a výstupy striktně validované vůči formálním JSON schémátům. Zajišťuje vysokou typovou bezpečnost, avšak přináší režii tokenů spotřebovaných na definice schémat.

4.2.9.1 Model Context Protocol

Model Context Protocol — an open standard originally introduced by Anthropic for standardized communication between AI applications and external tools, resources, and data through JSON-RPC messages. ⁽³¹⁾ Pro sjednocení rozhraní mezi AI aplikacemi a externími nástroji či datovými zdroji vznikl otevřený standard **MCP**: — [cz] Model Context Protocol — otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC. ⁽³¹⁾ Namísto vytváření proprietárních rozhraní pro každou službu definuje MCP standardizovaný způsob komunikace.

Základní vlastnosti protokolu MCP:

- Protokolové rozhraní: Komunikace probíhá prostřednictvím standardu JSON-RPC (přes standardní vstup/výstup `stdio` nebo proud událostí `Server-Sent Events / SSE`).
- Architektonické oddělení: Implementace nástrojů běží jako samostatný proces mimo jádro harnessu. MCP servery fungují jako znovupoužitelné komponenty, které lze snadno sdílet napříč různými agenty a projekty.

4.2.10 Skills

Reusable modular packages of instructions (typically defined in a SKILL.md file), procedural rules, and optional helper scripts or resources that a harness dynamically loads into an agent's context for a particular class of task. ⁽³²⁾ Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží - **Skills** : [+] **Skills** : ⁽³²⁾.

Architektura dovedností staví na následujících principech:

- Definiční soubor **SKILL.md**: Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.
-
- Dynamické načítání pro úsporu kontextu: Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Kompletní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- Skripty () a záchytné body (Hook): Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

Kromě kontextových dovedností využívají pokročilé řídicí architektury také programové **Plugins** : — [cz] Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body.. Zatímco *Skills* fungují jako kontextové procedury a instrukce interpretované modelem, plugíny rozšiřují samotný harness na nativní systémové úrovni. [+] Načítání dovedností popisuje samostatný koncept **Progressive Disclosure** :. Deterministické součásti dovedností jsou rozděleny mezi : a **Hook** : nativní rozšíření harnessu představuje samostatný koncept **Plugins** :.

4.2.10.1 Progressive Disclosure

A strategy where only compact capability descriptions are initially placed in active context and detailed instructions are loaded only when needed. ⁽³²⁾ U dovedností snižuje postupné zpřístupňování kontextovou režii: základní prompt obsahuje pouze přehled dostupných dovedností a úplný obsah příslušného **SKILL.md** se načte až tehdy, když jej agent pro konkrétní úlohu potřebuje.

4.2.10.2 Script

A file or sequence of commands intended for automated execution by an interpreter, shell, or another runtime. ⁽³²⁾ Skript poskytuje deterministickou exekuci pro úlohy, u nichž není vhodné znovu rozhodovat pomocí jazykového modelu, například pro opakovatelné transformace, validace nebo obslužné kroky dovedností.

4.2.10.3 Event Hook

A defined lifecycle or event point to which custom deterministic logic can be attached before, after, or in place of standard system behavior. ⁽²⁸⁾ Hook váže deterministickou logiku na konkrétní událost životního cyklu harnessu, například před spuštěním nástroje, po dokončení kroku nebo při změně stavu běhu.

4.2.11 Context Engineering

Kontextové inženýrství je systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu. Kontext zahrnuje systémové instrukce, pracovní historii, výsledky nástrojů, externě načtená data a další informace, které model používá při rozhodování. Návrh této vrstvy určuje, které informace jsou modelu dostupné, kdy se načítají, jak dlouho zůstávají aktivní a jak se nahrazují při omezené kapacitě kontextového okna. Kontextové inženýrství řídí informační prostředí modelu; cílem není maximalizovat objem kontextu, ale udržet relevantní, přesný a provozně použitelný stav.

4.2.11.1 Context Compaction

Kompakce kontextu je zmenšení aktivního kontextu výběrem, shrnutím nebo nahrazením starší historie kompaktnější reprezentací tak, aby se běh vešel do kontextového okna. Správa aktivního kontextu je součástí kontextového inženýrství (). Při rozsáhlejších úlohách se kontextové okno nevyhnutelně zaplní. V okamžiku, kdy objem historie dosáhne kritické hranice, musí agent harness přistoupit ke kompakci kontextu - (*compaction*) [*+*](*compaction* ⁽³³⁾) — model je vyzván, aby dosavadní průběh sezení zkrátil do syntetického souhrnu, který nahradí starší část historie.

Tento proces však představuje destruktivní ztrátovou kompresi:

- Ztráta deterministických detailů: Model při rekurzivním zkracování vynechává přesná čísla řádků, signatury privátních funkcí, přesné cesty k souborům a doslovná chybová hlášení kompilátoru.
- Oslabení negativních pravidel: Explicitní zákazy (např. neměnit veřejné rozhraní API) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.
- Konfirmační zkreslení (*Confirmation Bias*): Model v souhrnu upřednostňuje fakta odpovídající jeho vnitřním statistickým asociacím na úkor netriviálních specifik konkrétního projektu.

Kompakce prodlužuje běh za cenu ztráty informace, a proto musí být navržena tak, aby neodstraňovala detaily nutné pro další rozhodování.

4.2.11.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) je architektura, v níž systém před generováním nebo během něj vyhledá relevantní informace z externího zdroje a vloží je do kontextu modelu. Aby se předešlo ztrátě informací způsobené kompakcí, moderní agentní architektury přesouvají část paměti mimo samotné kontextové okno. Namísto spoléhání se na jediný lineární textový kontext se uplatňují strukturovaná externí úložiště.

K hlavním přístupům patří:

- - Hierarchická epizodická paměť (RAG) [+]Hierarchická epizodická paměť (RAG⁽³⁴⁾):

Ukládání doslovných protokolů nástrojů a historie úloh do externí databáze; do kontextu se selektivně injektují pouze bezprostředně relevantní fragmenty.

- Persistentní graf stavu projektu (*Project State Graph*): Udržování explicitního, strukturovaného přehledu o stavu repozitáře (seznam modifikovaných souborů, otevřené úkoly, výsledky testů a platné invarianty) mimo kontextové okno.

Díky tomu může agent kdykoliv obnovit přesný stav projektu bez závislosti na ztrátovém rekurzivním shrnování. RAG odděluje dlouhodobé uchování informací od omezeného aktivního kontextu a umožňuje načítat pouze data relevantní pro aktuální krok.

4.2.11.3 Context Rot

Degradace kontextu je pokles schopnosti modelu spolehlivě využívat informace v dlouhém, hlučném nebo vzájemně si konkurujícím kontextu. Schopnost jazykového modelu pracovat s dlouhým kontextem nelze posuzovat pouze podle nominální velikosti okna. Ačkoliv moderní modely deklarují kapacitu statisíců tokenů, jejich schopnost efektivně vyhledávat a logicky propojovat fakta s rostoucí délkou kontextu výrazně klesá. Tento jev se označuje jako : — .

V praxi se projevuje dvěma hlavními mechanismy:

- Lost in the Middle⁽²⁰⁾: Pozornostní vrstvy transformeru spolehlivě vnímají informace na samém začátku a konci okna, zatímco fakta umístěná uprostřed dlouhého textu jsou často přehlížena.
- Multi-Needle Reasoning: Schopnost logicky provázat několik na sobě závislých informací rozptýlených napříč různými soubory; s rostoucí délkou kontextu tato schopnost prudce klesá.

Při komplexním křížovém refaktoringu ve velkém kontextu proto model často přehlédne klíčové souvislosti, které by v menším a čistším okně zpracoval bez potíží. Nominálně větší kontextové okno samo o sobě nezaručuje lepší výkon; kvalita závisí na tom, zda model dokáže relevantní informace v kontextu skutečně najít a propojit.

4.2.11.4 Semantic Drift

Sémantický posun je postupné zkreslování významu a faktického stavu při opakovaném ztrátovém shrnování nebo transformaci kontextu. Pokud je historie sezení v dlouhém vývojovém běhu shrnována vícekrát po sobě, vzniká řetězec ztrátových transformací ($S_{k+1} = f(S_k, \Delta_k)$).

Rizika sémantického posunu spočívají v těchto jevech:

- Efekt tiché pošty: Drobné zkreslení či halucinace vzniklá v kole k je v kole $k + 1$ přijata jako nezpochybnitelný historický fakt.
- Divergence modelu od reality: Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému.

Výsledkem je stav, kdy agent sebevědomě reportuje vyřešení úkolu, ačkoliv reálný kód zůstává v nefunkčním či neúplném stavu. Opakovaná komprese může změnit pracovní

reprezentaci reality natolik, že další rozhodování vychází z chybných historických předpokladů.

4.2.12 Workflow-graph Engineering

Inženýrství pracovních grafů je návrh agentních nebo automatizačních workflow jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky. Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

- Klíčové přístupy ke škálování zahrnují:

- Subagenti (*Subagents*⁽³⁵⁾): Hlavní orchestrátor dekomponuje rozsáhlou úlohu a deleguje dílčí kroky na specializované agenty (např. průzkumník, repozitáře, plánovač, kódovací dělník). Po dokončení je kontext subagenta zahozen a orchestrátor obdrží pouze čistý výsledek, což chrání primární kontext před znečištěním (*context pollution*).
- Pracovní postupy jako grafy (DAG / Graph Engineering): Životní cyklus požadavku je modelován jako orientovaný acyklický graf (příjem → plán → kód → testy → schválení). Hrany definují striktní závislosti (*needs*); selhání v libovolném uzlu okamžitě zastaví navazující kroky.

[+] Škálování je dále rozloženo na dva samostatné koncepty: hierarchickou delegaci prostřednictvím *Subagent* : a explicitní závislosti pracovního postupu prostřednictvím *DAG* :

Grafová struktura umožňuje rozdělit složitou úlohu na kontrolovatelné kroky, oddělit jejich odpovědnosti a explicitně řídit závislosti mezi nimi.

4.2.12.1 Subagent

Subagent je dočasná nebo specializovaná agentní instance, které nadřazený orchestrátor deleguje vymezenou dílčí úlohu a následně převezme její výsledek. Při hierarchické dělbě práce hlavní orchestrátor rozděluje rozsáhlou úlohu a jednotlivé části deleguje specializovaným subagentům, například pro průzkum repozitáře, plánování nebo implementaci. Po dokončení dílčího běhu může nadřazený agent převzít pouze jeho výsledek namísto celé pracovní historie subagenta. Subagenti umožňují paralelní nebo specializovanou práci a současně omezují množství dílčí pracovní historie, které musí zůstat v kontextu hlavního orchestrátoru.

4.2.12.2 Directed Acyclic Graph

Orientovaný acyklický graf (DAG) je orientovaný graf bez orientovaného cyklu, který umožňuje explicitně vyjádřit závislosti a pořadí kroků pracovního postupu. V agentním workflow může DAG modelovat kroky jako uzly a jejich povinné závislosti jako hrany. Tím lze explicitně vyjádřit například posloupnost příjem požadavku → plán → implementace → testy → schválení a zabránit spuštění navazujícího kroku před splněním jeho předpokladů.

DAG převádí implicitní pořadí vícefázové úlohy na kontrolovatelnou strukturu závislostí, v níž lze navazující krok spustit až po splnění jeho předpokladů.

5 Results and Discussion

6 Conclusion

 **Metodické vymezení / Rozsah práce:** Poznámka k vypracování závěru: Závěr práce bude sepsán jako poslední krok po definitivním ucelení teoretických východisek agentického inženýrství a detailní architektury agent harnessu. Tato závěrečná kapitola syntetizuje zjištění o deterministickém řízení autonomních agentů a zhodnotí formulované principy a výzkumné otázky bez vazby na dřívější ad-hoc testovací repozitáře.

List of figures and tables

Obrázek 1	AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute.	8
Obrázek 2	Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.	16
Obrázek 3	GPT-5.6. Zdroj: OpenAI.	17
Obrázek 4	Claude Opus 5. Zdroj: Anthropic.	18
Obrázek 5	DeepSeek-V4.1-Flash. Zdroj: DeepSeek.	18
Obrázek 6	Aplikace Codex. Zdroj: OpenAI.	22
Obrázek 7	Claude Code při práci se souborem. Zdroj: Anthropic.	23
Obrázek 8	Aplikační rozhraní Claude. Zdroj: Anthropic.	24
Obrázek 9	<u>Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.</u>	27

List of appendices

References

1. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion: Q1 2026 Trends and Insights. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
2. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
3. ANTHROPIC. Claude Code. Online. 2026. [Accessed 20 září 2026]. Available from: <https://claude.com/product/claude-code>
4. MICROSOFT AI ECONOMY INSTITUTE. Global AI Adoption in 2025—A Widening Digital Divide. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/corporate-responsibility/topics/ai-economy-institute/reports/global-ai-adoption-2025/>
5. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
6. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.
7. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
8. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
9. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Proceedings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
10. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
11. OPENAI. ChatGPT macOS app—Screenshot Tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://help.openai.com/en/articles/9295245-chatgpt-macos-app-screenshot-tool>
12. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.
13. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of*

- Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
14. OPENAI. GPT-5.6: Frontier intelligence that scales with your ambition. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/gpt-5-6/>
 15. ANTHROPIC. Introducing Claude Opus 5. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.anthropic.com/news/claude-opus-5>
 16. DEEPSEEK. Introducing DeepSeek-V4.1-Flash: smarter, faster, more efficient. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.deepseek.com/en/news/deepseek-v4-1-flash/>
 17. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
 18. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
 19. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
 20. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
 21. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
 22. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
 23. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
 24. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

25. ANTHROPIC. Navigating the Claude desktop app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://academy.claude.com/tutorials/navigating-the-claude-desktop-app>
26. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
27. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
28. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
29. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
30. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
31. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
32. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
33. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMlingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
34. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
35. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan,

- WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
36. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>